

STL : 函数对象和智能指针 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/hml>

课程团队：刘知远 任炬 黄民烈

上期要点回顾

- 命名空间
- STL初步——容器与迭代器
- `string`字符串类

本讲内容提要

- **iostream**输入输出流
- 函数对象
- 智能指针与引用计数
- 字符串处理与正则表达式（自学）

iostream

输入输出流

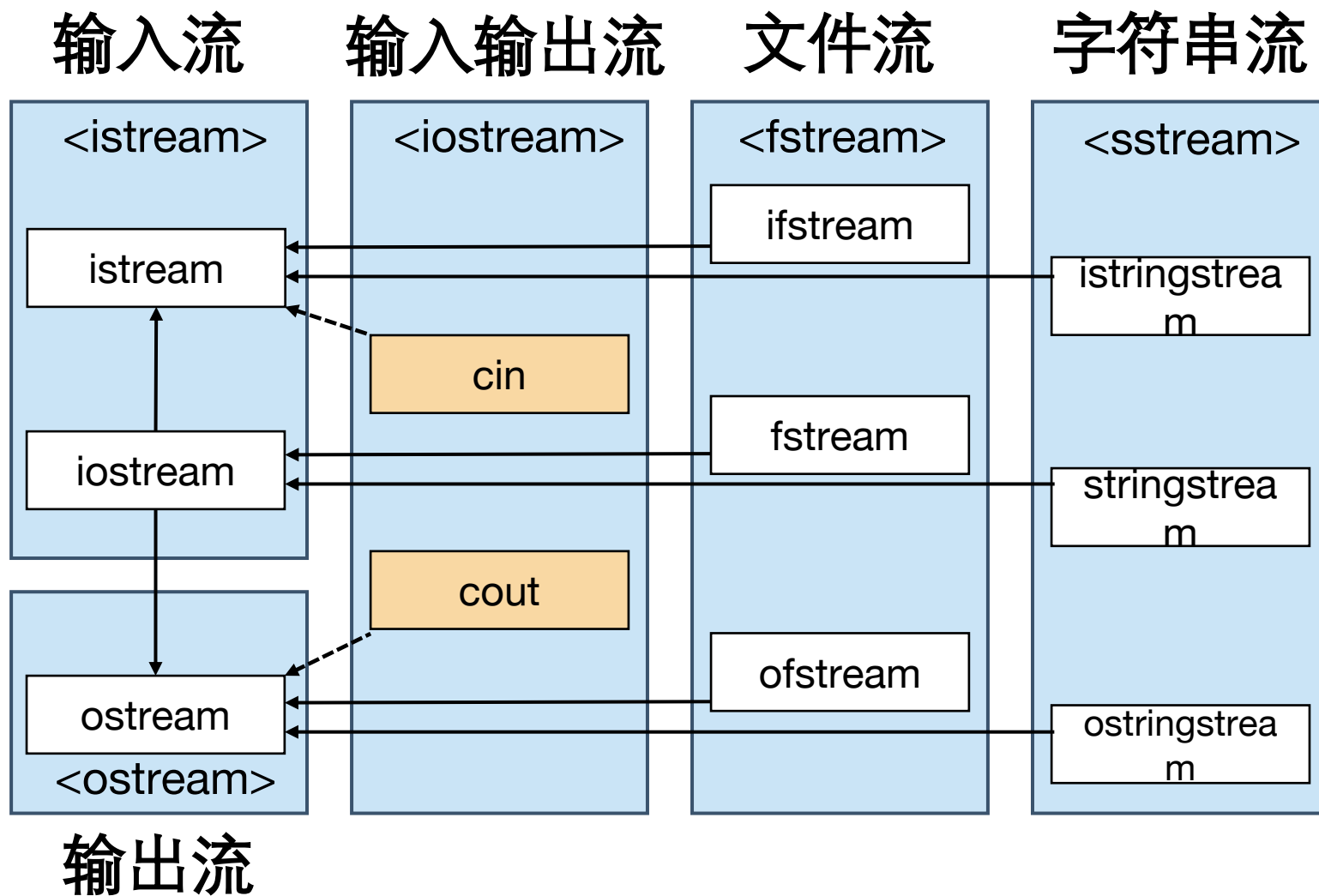
回忆：重载输出流运算符

```
cout << str << endl;  
cin >> str;
```

```
ostream& operator<<(ostream& out, const Test& src)  
{  
    out << src.id << endl;  
    return out;  
}
```

■ **ostream**到底是什么？

STL输入输出流



从ostream和cout开始

- **ostream**即**output stream**

是**STL**库中所有**输出流**的**基类**

- 它重载了针对**基础类型**的输出流运算符 (**<<**)

接受不同类型的数据，再调用系统函数进行输出

- 统一了输出**接口**，改善了**C**中输出方式混乱的状况

- **printf("%d %f %s", 1, 2.3, "hello");**

<http://www.cplusplus.com/reference/ostream/ostream/operator%3C%3C/>

- **cout**是**STL**中内建的一个**ostream**对象

- 它会将数据送到**标准输出流**（一般是屏幕）

实现自己的ostream

```
class ostream
```

```
{
```

```
public:
```

```
    ostream& operator<<(char c)
```

```
{
```

```
    printf("%c", c);
```

```
    return *this;
```

```
}
```

```
    ostream& operator<<(const char* str)
```

```
{
```

```
    printf("%s", str);
```

```
    return *this;
```

```
}
```

```
}cout;
```

```
int main()
```

```
{
```

```
    cout << "hello" << ' '
```

```
    << "world";
```

```
    return 0;
```

```
}
```

实现原理:

<<运算符为左结合

先执行cout << "hello" 调用第二个函数 返回c1 (cout的引用)

再执行c1 << ' ' 调用第一个函数 返回c2 (cout的引用)

最后执行c2 << "world" 调用第二个函数

格式化输出

■ 如何格式化输出 — #include <iomanip>

```
cout << fixed << 2018.0 << " " << 0.0001 << endl;  
      //浮点数 -> 2018.000000 0.000100  
cout << scientific << 2018.0 << " " << 0.0001 << endl;  
      //科学计数法 -> 2.018000e+03 1.000000e-04  
cout << defaultfloat; //还原默认输出格式  
cout << setprecision(2) << 3.1415926 << endl;  
      //输出精度设置为2 -> 3.2  
cout << oct << 12 << " " << hex << 12 << endl;  
      //八进制输出 -> 14 十六进制输出 -> c  
cout << dec; //还原十进制  
cout << setw(3) << setfill('*') << 5 << endl;  
      //设置对齐长度为3, 对齐字符为* -> **5
```

格式化输出

■ 以 `setprecision` 为例

- `cout << setprecision(2) << 1.05 << endl;`
- 保留2位精度，输出1.1

■ 如何实现？

- C++标准中未定义，不同编译器有自己的实现方式
- 一种实现方式的示例

```
class setprecision
{
private:
    int precision;
public:
    setprecision(int p) : precision(p) {}
    friend class ostream;
};
// setprecision(2) 是一个类的对象
```

流操纵算子(stream manipulator)

```
class ostream
{
private:
    int precision; //记录流的状态
public:
    ostream& operator<<
        (const setprecision &m) {
        precision = m.precision;
        return *this;
    }
} cout;
```

```
cout << setprecision(2);
// setprecision(2) 是一个类的对象
```

- 借助辅助类，设置成员变量
- 这种类叫流操纵算子

流操纵算子：endl

■ C++标准中endl的声明

- ostream& endl(ostream& os);

■ endl是一个函数

- 等同于输出'\n'，再清空缓冲区 os.flush()

```
ostream& endl(ostream& os) {  
    os.put('\n');  
    os.flush();  
    return os;  
}
```

- 可以调用 endl(cout);

■ 缓冲区

- 目的是减少外部读写次数
- 写文件时，只有清空缓冲区或关闭文件才能保证内容正确写入

流操纵算子：endl

■ **endl** 同时也是流操纵算子，如何实现？

- `cout << endl;`

■ 一种实现方式的示例

```
ostream& operator<<
    (ostream& (*fn)(ostream&)) {
    //流运算符重载，函数指针作为参数
    return (*fn)(*this);
}
```

不能复制的cout

■ 注意重载流运算符的方式

`ostream& operator<<(const char &c)`

`friend ostream& operator<<(ostream& os, MyClass obj)`

■ 为什么重载流运算符要返回引用?

- 避免复制

■ 观察ostream的复制构造函数

- `ostream(const ostream&) = delete;`
- `ostream(ostream&& x);`
- 禁止复制、只允许移动
- 仅使用cout一个全局对象

不能复制的cout

■ 为什么只能使用一个对象?

- 减少复制开销
- 一个对象对应一个标准输出，符合OOP思想
- 多个对象之间无法同步输出状态

■ 是否能做得更好?

- 全局对象往往引入初始化顺序问题
- 单件模式 (Singleton Pattern)
- 在之后的设计模式中会介绍

文件输入输出流

- 以文件输入流作为例子
- **ifstream**是**istream**的子类
- 功能是从文件中读入数据

■ 打开文件

- `ifstream ifs("input.txt");`
- `ifstream ifs("binary.bin", ifstream::binary);`
`// 以二进制形式打开文件`
- `ifstream ifs;`
`ifs.open("file")`
`// do something`
`ifs.close()`

读入示例

```
#include <iostream>
#include <string>
#include <cctype>
#include <fstream>
using namespace std;

int main() {
    ifstream ifs("input.txt");
    while(ifs) { //判断文件是否到末尾 利用了重载的bool运算符
        ifs >> ws; //除去前导空格 ws也是流操纵算子
        int c = ifs.peek(); //检查下一个字符, 但不读取
        if (c == EOF) break;
        if (isdigit(c)) //<cctype>库函数
        {
            int n;
            ifs >> n;
            cout << "Read a number: " << n << endl;
        } else {
            string str;
            ifs >> str;
            cout << "Read a word: " << str << endl;
        }
    }
    return 0;
}
```

其他操作

■ `getline(cin, str)`

- `ifstream` 是 `istream` 的子类
- 故 `getline(ifs, str)` 仍然有效

■ 其他操作

- `get()` 读取一个字符
- `ignore(int n=1, int delim=EOF)`
丢弃 `n` 个字符，或者直至遇到 `delim` 分隔符
- `peek()` 查看下一个字符
- `putback(char c)` 返还一个字符
- `unget()` 返还一个字符
-

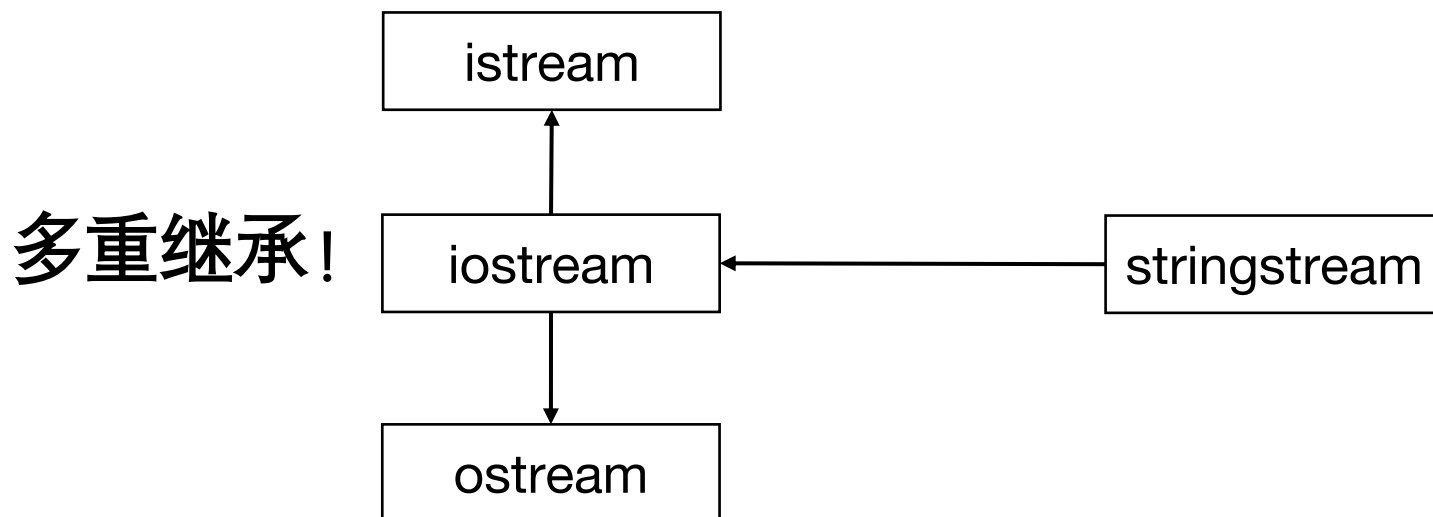
istream与scanf

■ 为什么C++使用流输入取代了scanf

- scanf不友好，不同类型要使用不同的标识符
`scanf("%d %hd %f %lf %s", &i, &s, &f, &d, name);`
`cin >> i >> s >> f >> d >> name;`
- 安全性
`scanf("%d", &a);` //可能写入非法内存
- 可拓展性
`MyClass obj;`
`cin >> obj;`
- 性能
scanf在运行期间需要对格式字符串进行解析
istream在编译期间已经解析完毕

字符串输入输出流

- 以输入输出流作为例子
- **stringstream**是**iostream**的子类
- **iostream**继承于**istream**和**ostream**
- **stringstream**实现了输入输出流双方的接口



stringstream

■ stringstream

- 它在对象内部维护了一个buffer
- 使用流输出函数可以将数据写入buffer
- 使用流输入函数可以从buffer中读出数据

■ 一般用于程序内部的字符串操作

■ 构造方式

- `stringstream ss; // 空字符串流`
- `stringstream ss(str); // 以字符串初始化流`

使用示例

```
#include <sstream>
using namespace std;

int main() {
    stringstream ss;
    ss << "10";
    ss << "0 200";

    int a, b;
    ss >> a >> b;    //a=100 b=200
    return 0;
}
```

- 可以连接字符串
- 可以将字符串转换为其他类型的数据
- 配合流操作算子，可以达到格式化输出效果

获取stringstream的buffer

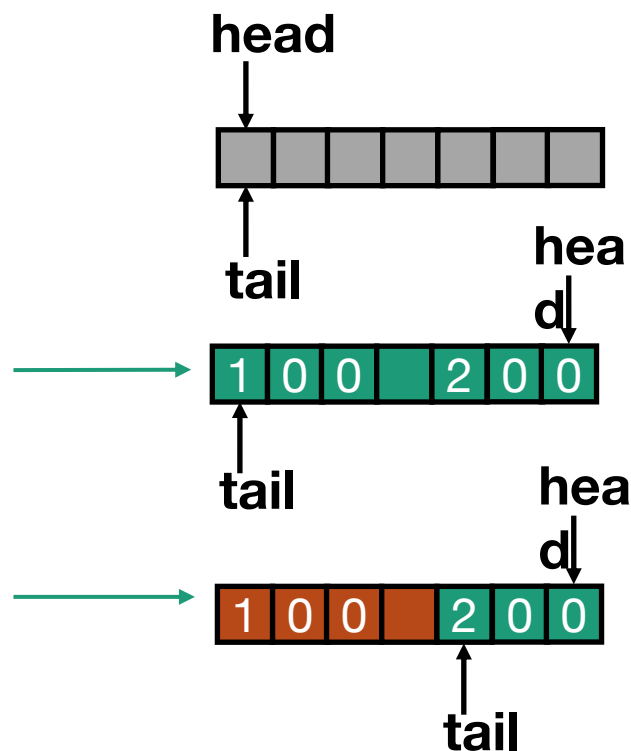
■ ss.str()

- 返回一个string对象
- 内容为stringstream的buffer

■ 注意buffer内容并不是未读取的内容

```
#include <sstream>
#include <iostream>
using namespace std;
```

```
int main() {
    stringstream ss;
    ss << "100 200";
    cout << ss.str() << endl;
    // "100 200"
    int a, b;
    ss >> a; // a = 100
    cout << ss.str() << endl;
    // "100 200"
    ss >> b; // b = 200
    return 0;
}
```



head和tail
间代表未读
取的部分

实现一个类型转换函数

■ 如何实现字符串与整数的互相转换?

- to_string 转换为字符串
- stoi 转换为整数

■ 其他类型呢? 可以使用一个函数实现吗?

```
#include <iostream>
```

```
#include <sstream>
```

```
using namespace std;
```

```
int main() {
```

```
    string x = convert<string>(123);
```

```
    int y = convert<int>("456");
```

```
    cout << x << endl;
```

```
    cout << y << endl;
```

```
    return 0;
```

```
}
```

实现一个类型转换函数

```
template<class outtype, class intype>
outtype convert(intype val)
{
    static stringstream ss;
                                //使用静态变量避免重复初始化
    ss.str("");                //清空缓冲区
    ss.clear();                //清空状态位 (不是清空内容)
    ss << val;
    outtype res;
    ss >> res;
    return res;
}
```

关于状态位：状态位记录流的状态，例如是否读入了非法字符
<http://www.cplusplus.com/reference/ios/ios/setstate/>

函数对象

回忆：什么是函数

■ 看一个例子

- 如果flag是1，则对每个元素调用inc；否则调用dec

```
#include <iostream>
```

```
int arr[5] = { 5, 2, 3, 1, 7 };
```

```
void increase(int &x){x++;}
```

```
void decrease(int &x){x--;}
```

```
int main()
```

```
{
```

```
    int flag; std::cin >> flag;
```

```
    if (flag == 1) {
```

```
        for (int &x : arr) { increase(x);}
```

```
    }else{
```

```
        for (int &x : arr) { decrease(x);}
```

```
    }
```

```
    return 0;
```

```
}
```

- 仅仅只有调用的函数不同，如何减少重复的逻辑？

回忆：什么是函数

■ 可以使用变量表示函数吗？

```
if (flag == 1)
    func = increase;
else
    func = decrease;
for (int &x : arr) { func(x);}
```

■ 当然可以。在C中，func是指向函数的指针。

• `void (*func)(int&);` // 函数指针的声明

返回值

参数列表

指针符号 声明的变量名

回忆：什么是函数

- 函数的类型比较难写，使用**auto**可以自动推断

//使用**auto**时必须初始化

```
auto func = flag==1?increase:decrease;  
for (int &x : arr) { func(x);}
```

- auto自动推断出func的类型为void (*)(int&);

- 和数组类似：

- 数组名 = 指向数组第一个元素的指针
- 函数名 = 指向函数的指针

回忆：什么是函数

```
#include <iostream>
using namespace std;
void increase(int &x){x++;} void decrease(int &x){x--;}
int main()
{
    int flag; cin >>flag;

    int arr[] = {1,2,3,4,5};
    void (*func)(int&); //声明函数指针
    if(flag==1) {func=increase; } else {func=decrease;}
    //auto func = flag==1?increase:decrease; //和上两行效果相同

    for (int &x:arr) { func(x); cout << x;}

    return 0;
}
```

函数作为变量

■ 例子： 给一个长度为n的数组，如何排序？

- `std::sort` 来自 `<algorithm>`
- `template<class Iterator>`
`void sort (Iterator first, Iterator last);`

```
#include <algorithm>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int arr[5] = { 5, 2, 3, 1, 7 };
```

```
    std::sort(arr, arr + 5);
```

```
    for (int x : arr) { cout << x << " " ;}
```

```
    // 1 2 3 5 7
```

```
    return 0;
```

```
}
```

函数作为变量

- 如果想倒转排序?
- 注意到**sort**还重载了另一套参数

```
template <class Iterator, class Compare>  
void sort (Iterator first,  
           Iterator last, Compare comp);
```

比较函数comp:

```
bool comp(int a, int b)  
{  
    return a > b;  
    //comp函数传入两个值  
    //若a在b前, 则返回true  
    //若a在b后 或 a等于b, 则返回false  
}
```

函数作为变量

```
#include <algorithm>
#include <iostream>
using namespace std;

bool comp(int a, int b)
{   return a > b; }

int main(){
    int arr[5] = { 5, 2, 3, 1, 7 };
    std::sort(arr, arr + 5, comp);
    for (int x : arr) {
        cout << x << " ";
    } // 7 5 3 2 1
    return 0;
}
```

函数作为变量

■ 实际上，**Compare**就是**comp**的类型

- 函数指针：bool (*)(int,int)

■ STL提供了预定义的比较函数(#include <functional>)

- 从小到大

sort(arr, arr+5, less<int>())

- 从大到小

sort(arr, arr+5, greater<int>())

■ 疑问：

- 对比 sort(arr, arr + 5, comp)
- greater<int>()为什么带括号？是什么？

函数对象

■ 实际上，`greater<int>()`是一个对象

- `greater`是一个模板类
- `greater<int>` 用`int`实例化的类
- `greater<int>()` 该类的一个对象

■ 同时，它表现的像一个函数

```
#include <functional>
#include <iostream>
using namespace std;
int main() {
    auto func = greater<int>();
    cout << func(2, 1) << endl; //True
    cout << func(1, 1) << endl; //False
    cout << func(1, 2) << endl; //False
    return 0;
}
```

■ 因此，这种对象被称为函数对象

如何实现函数对象

```
#include <iostream>
using namespace std;
template<class T>
class Greater {
public:
```

■ 注意三个const

- 排序中，comp不能修改数据
- 一般情况下，comp也不应该修改自身

```
    bool operator()(const T &a, const T &b) const{
        return a > b;
    }
};
int main(){
    auto func = Greater<int>();
    cout << func(2, 1) << endl; //True
    cout << func(1, 1) << endl; //False
    cout << func(1, 2) << endl; //False
    return 0;
}
```

如何实现函数对象

■ 函数对象的要求有哪些？

- 需要重载operator()运算符
- 并且该函数需要是public访问权限

■ 小知识： Duck Typing 鸭子类型

- 如果一个物体，叫声像鸭子、走路像鸭子，那么它就是鸭子；
- 如果一个对象，用起来像函数，那么它就是函数对象！
- C++没有严格定义什么是函数对象
- 但是实践上按Duck Typing来处理

实现自己的sort

```
bool comp(int a, int b)
{
    return a > b;
}

//sort(arr, arr+5, comp);
```

```
template<class T>
class greater
{
public:
    bool operator()(.....)
};

//sort(arr, arr+5, greater<int>())
```

■ sort的第三个参数是什么类型?

```
template <class Iterator, class Compare>
void sort (Iterator first, Iterator last,
          Compare comp);
```

- 模板类型，可以接受函数指针/函数对象

实现自己的sort

```
#include <algorithm>
#include <functional>
using namespace std;
```

```
bool comp(int a, int b)
{
    return a > b;
}
```

```
template<class Iterator, class Compare>
```

```
void mysort(Iterator first, Iterator last, Compare comp)
```

```
//mysort的时间复杂度为 $O(n^2)$ , std::sort的时间复杂度为 $O(n\log n)$ 
```

```
{
    for (auto i = first; i != last; i++)
        for (auto j = i; j != last; j++)
            if (!comp(*i, *j)) swap(*i, *j);
}
```

```
int main()
```

```
{
    int arr[5] = { 5, 2, 3, 1, 7 };
    mysort(arr, arr + 5, comp);
    mysort(arr, arr + 5, greater<int>());
    //既可接受函数指针, 又可接受函数对象
    return 0;
}
```

自定义类型的排序

- 假设有一个 **class People**

```
class People
{
public:
    int age, weight;
};
```

- 如何按照年龄从小到大排序?

自定义类型的排序

■ 方法一：重载小于运算符

```
#include <algorithm>
#include <vector>
using namespace std;
class People
{
public:
    int age, weight;
    bool operator<(const People &b) const
    {
        return age < b.age;
    }
};
int main() {
    vector<People> vec = {{18, 50}, {16, 40}};
    sort(vec.begin(), vec.end());
    return 0;
}
```

People的operator<
一定按年龄计算吗？
体重怎么办？

自定义类型的排序

■ 方法二：定义比较函数

```
#include <algorithm>
#include <vector>
using namespace std;
```

```
class People
{ public: int age, weight; };
```

```
bool compByAge(const People &a, const People &b)
{   return a.age < b.age; }
```

```
int main() {
    vector<People> vec = {{18, 50}, {16, 40}};
    sort(vec.begin(), vec.end(), compByAge);
    return 0;
}
```

自定义类型的排序

■ 方法三：定义比较函数对象

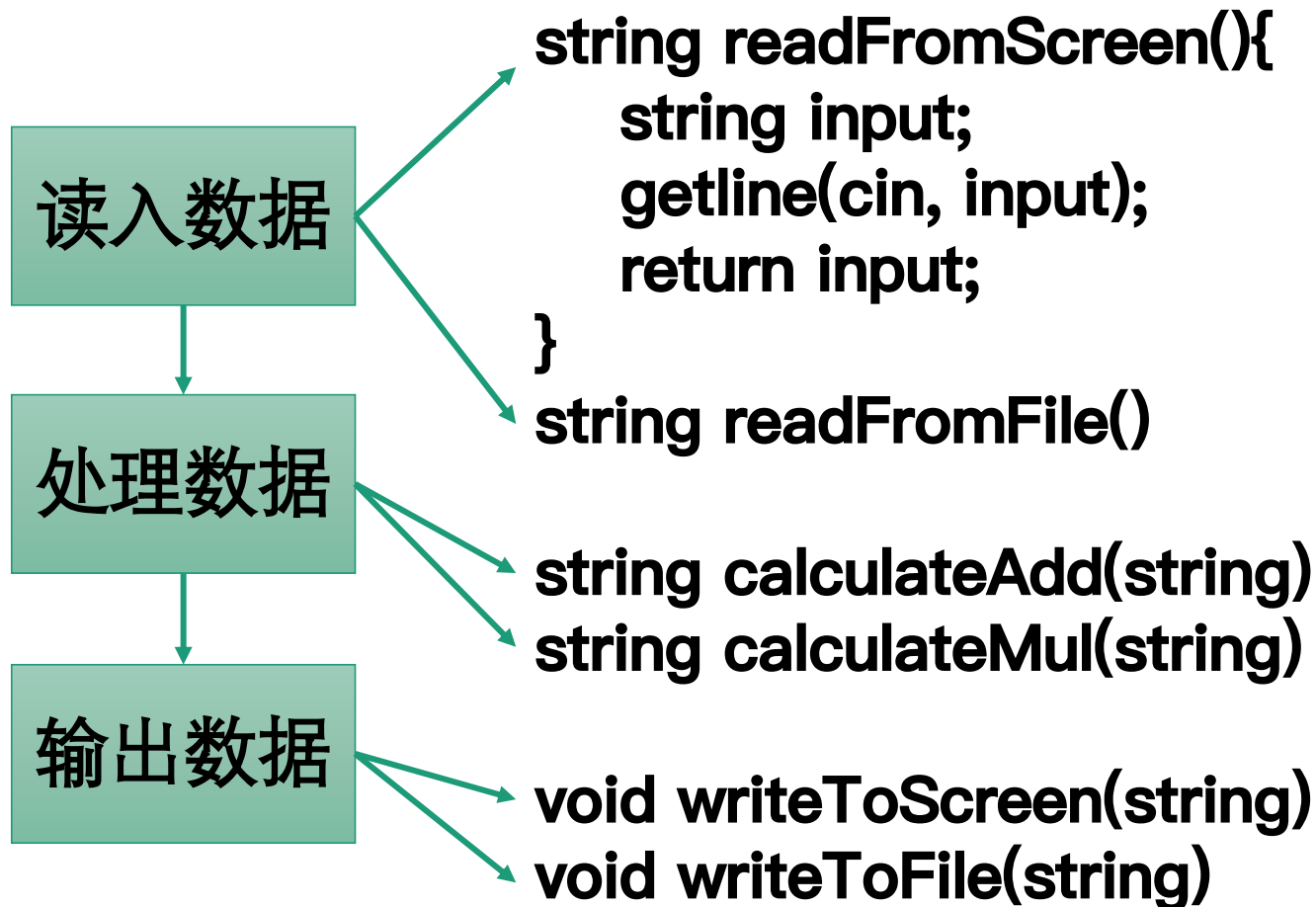
```
#include <algorithm>
#include <vector>
using namespace std;
class People
{ public:    int age, weight; };

class AgeComp {
public:
    bool operator()(const People &a, const People &b) const
    {    return a.age < b.age;    }
};

int main() {
    vector<People> vec = {{18, 50}, {16, 40}};
    sort(vec.begin(), vec.end(), AgeComp());
    return 0;
}
```

例子：一个简单计算器

- 分为三个步骤，每个步骤都有可选的方式



例子：一个简单计算器

■ 基于虚函数的模板 (Template) 设计模式

L9课件提到过

```
class CalculatorBase
{
public:
    virtual string read();
    virtual string calculate(string);
    virtual void write(string);
    void process()
    {
        string data = read();
        string output = calculate(data);
        write(output);
    }
};
```

例子：一个简单计算器

- 如果使用函数对象？能不能写成

```
void process(ReadFunc read,  
             CalFunc calculate,  
             WriteFunc write)  
{  
    string data = read();  
    string output = calculate(data);  
    write(output);  
}  
process(readFromScreen,  
        calculateAdd, writeToFile);
```

- ReadFunc, CalFunc, WriteFunc分别是什么？

例子：一个简单计算器

■ 如果参数只有函数指针

- ReadFunc = string(*) (void)
- CalFunc = string(*) (string)
- WriteFunc = void(*) (string)

■ 但假设参数还可能有函数对象怎么办？

■ 例如

```
class ReadFromFile {  
public:  
    string operator()(){  
        string input;  
        getline(ifstream("input.txt"), input);  
        return input;  
    }  
};
```

使用模板函数

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
//省略readFromScreen/ReadFromFile/calculateAdd/writeToScreen
```

```
template<class ReadFunc, class CalFunc, class WriteFunc>
void process(ReadFunc read, CalFunc calculate, WriteFunc write)
{
    string data = read();
    string output = calculate(data);
    write(output);
}

int main()
{
    process(readFromScreen, calculateAdd, writeToScreen);
    process(ReadFromFile(), calculateAdd, writeToScreen);
    return 0;
}
```

完整代码上传到网络学堂

例子：一个简单计算器

■ 想用数组储存选项

```
auto readArr[] = {readFromScreen,  
                  ReadFromFile()};  
process(readArr[0], calculate, write);  
process(readArr[1], calculate, write);
```

■ auto 是什么类型？

- 无法推导！
- 函数指针和函数对象不是同一种类型

■ 需要一个类型能够统一两者

std::function类

- **std::function**类，来自<functional>头文件

返回值 参数列表

```
function<string()> readArr[] =  
    {readFromScreen, ReadFromFile()};  
function<string(string)> calculateArr[] =  
    {calculateAdd, CalculateMul()};  
function<void(string)> writeArr[] =  
    {writeToScreen, WriteToFile()};
```

- **function**为函数指针与对象提供了统一的接口

std::function类

```
#include <iostream>
#include <fstream>
#include <functional>
using namespace std;
```

```
//省略readFromScreen/ReadFromFile
```

```
int main()
```

```
{
```

```
    function<string()> readArr[] =
        {readFromScreen, ReadFromFile()};
```

```
    function<string()> readFunc;
```

```
    readFunc = readFromScreen; //允许函数的赋值
```

```
    readFunc = ReadFromFile();
```

```
    string (*readFunc2)();
```

```
    readFunc2 = readFromScreen;
```

```
    //readFunc2 = ReadFromFile(); //错误，类型不一致
```

```
    return 0;
```

```
}
```

使用function

```
#include <iostream>
#include <fstream>
#include <functional>
using namespace std;
```

```
//省略readFromScreen\ReadFromFile\calculateAdd\writeToScreen
```

```
void process(
    function<string()> read,
    function<string(string)> calculate,
    function<void(string)> write)
{
    string data = read();
    string output = calculate(data);
    write(output);
}

int main()
{
    process(readFromScreen, calculateAdd, writeToScreen);
    process(ReadFromFile(), calculateAdd, writeToScreen);
    return 0;
}
```

对比几种实现方式

■使用虚函数实现

- 需要构造基类和子类
- 运行时确定调用函数的地址

■使用模板实现

- 可以支持函数指针和函数对象
(通过模板, 自动重载实现)
- 编译期确定调用函数的地址
(当T不为std::function时)

■使用std::function实现

- 也可以支持函数指针和函数对象
(通过function的多态)
- 运行时确定调用函数的地址

std::function的意义

■ 函数对象化

- 万物皆对象，符合OOP的设计理念
- 函数可以作为参数传递
- 函数可以作为变量储存

■ 解决Duck Typing的繁琐问题

- 不再需要模板来调用不同的函数
- 简化理解，所有的函数都可以看做std::function

STL与函数对象

■ STL有大量函数用到了函数对象(#include <algorithm>)

- for_each 对序列进行指定操作
- find_if 找到满足条件的对象
- count_if对满足条件的对象计数
- binary_search 二分查找满足条件的对象
-

■ 并且也有许多预置的函数对象(#include <functional>)

- less 比较 $a < b$
- equal_to 比较 $a == b$
- greater 比较 $a > b$
- plus 返回 $a + b$
-

■ 熟练使用函数对象有助于实现复杂的功能

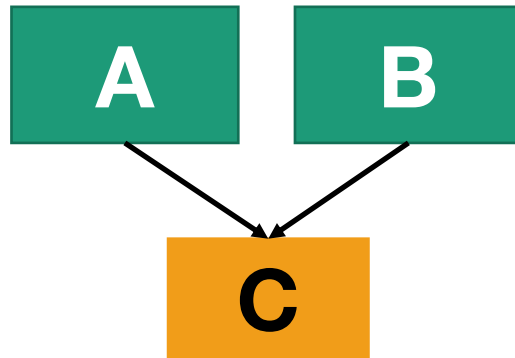
智能指针与 引用计数

指针的销毁

■ 一个例子：

- A、B对象共享一个C对象
- C对象不想交由外部销毁

■ A、B中的谁负责销毁C？



■ 应该在A、B都销毁时C才能销毁

■ 如何自动的处理？

智能指针

■ `shared_ptr` 来自 `<memory>` 库

- 构造方法

```
shared_ptr<int> p1(new int(1));  
shared_ptr<MyClass> p2 = make_shared<MyClass>(2);  
shared_ptr<MyClass> p3 = p2;  
shared_ptr<int> p4;    //空指针
```

- 访问对象

```
int x = *p1;    //从指针访问对象  
int y = p2->val; //访问成员变量
```

- 销毁对象

p2和p3指向同一对象，当两者均出作用域才会被销毁

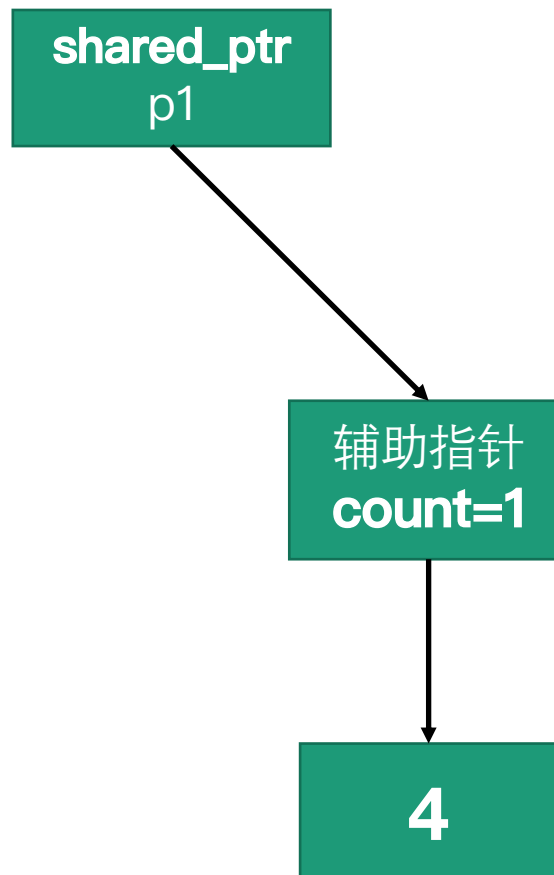
引用计数

- 为什么智能指针能够知道何时销毁对象?
- 引用计数! 当引用计数归0时, 销毁对象

```
#include <memory>
#include <iostream>
using namespace std;
int main()
{
    shared_ptr<int> p1(new int(4));
    cout << p1.use_count() << ' '; // 1
    {
        shared_ptr<int> p2 = p1;
        cout << p1.use_count() << ' '; // 2
        cout << p2.use_count() << ' '; // 2
    } //p2出作用域
    cout << p1.use_count() << ' '; // 1
}
```

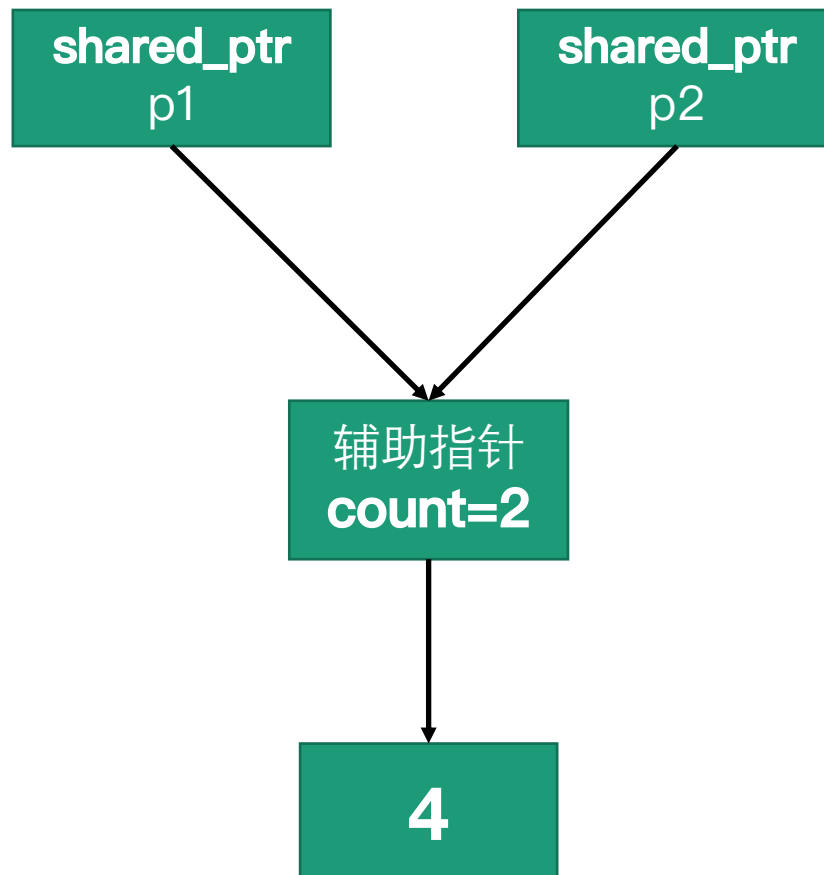
运行过程

```
shared_ptr<int> p1(new int(4));  
cout << p1.use_count() << ' ';  
{  
    shared_ptr<int> p2 = p1;  
    cout << p1.use_count() << ' ';  
    cout << p2.use_count() << ' ';  
} //p2出作用域  
cout << p1.use_count() << ' ';
```



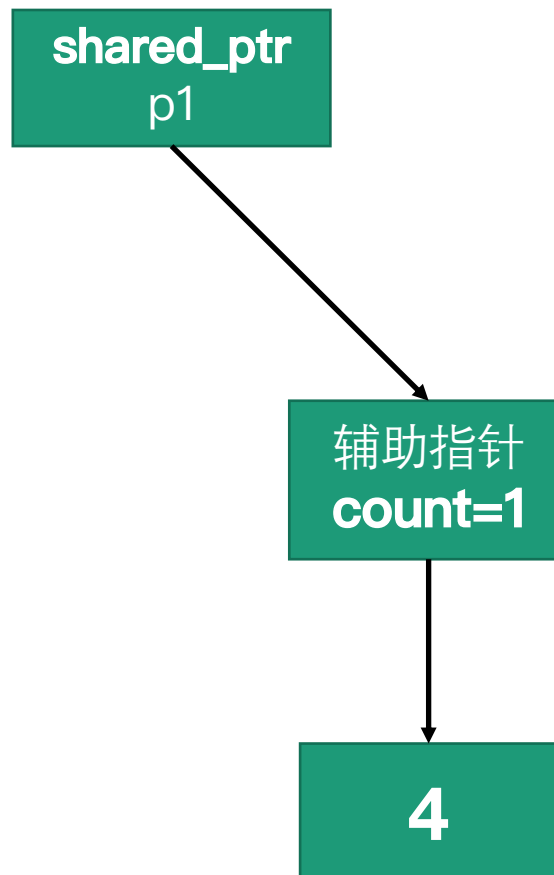
运行过程

```
shared_ptr<int> p1(new int(4));  
cout << p1.use_count() << ' ';  
{  
    shared_ptr<int> p2 = p1;  
    cout << p1.use_count() << ' ';  
    cout << p2.use_count() << ' ';  
} //p2出作用域  
cout << p1.use_count() << ' ';
```



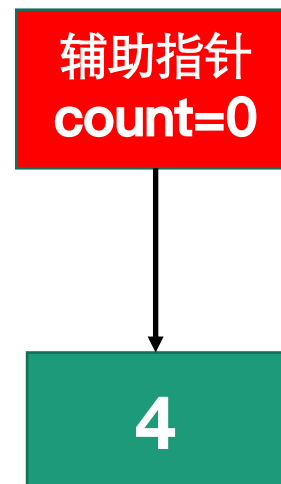
运行过程

```
shared_ptr<int> p1(new int(4));  
cout << p1.use_count() << ' ';  
{  
    shared_ptr<int> p2 = p1;  
    cout << p1.use_count() << ' ';  
    cout << p2.use_count() << ' ';  
} //p2出作用域  
cout << p1.use_count() << ' ';
```



运行过程

```
shared_ptr<int> p1(new int(4));  
cout << p1.use_count() << ' ';  
{  
    shared_ptr<int> p2 = p1;  
    cout << p1.use_count() << ' ';  
    cout << p2.use_count() << ' ';  
} //p2出作用域  
cout << p1.use_count() << ' ';  
//调用delete, 销毁int*
```



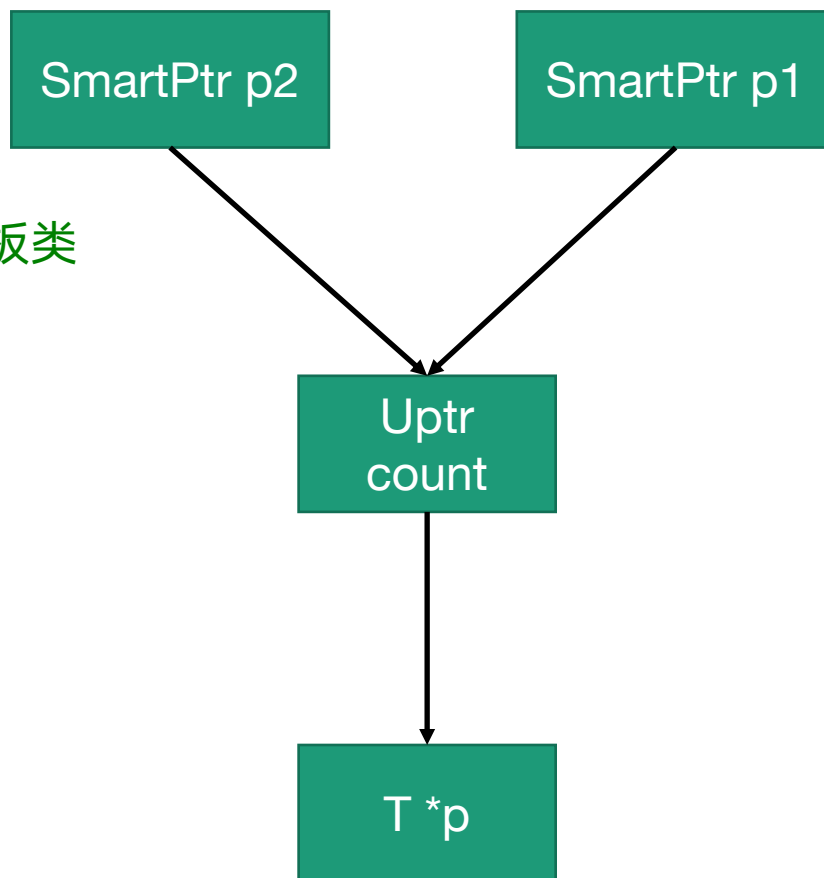
实现自己的引用计数

```
#include <iostream>
using namespace std;

template <typename T>
class SmartPtr; //声明智能指针模板类

template <typename T>
class U_Ptr { //辅助指针
private:
    friend class SmartPtr<T>;
    //SmartPtr是U_Ptr的友元类
    U_Ptr(T *ptr) :p(ptr), count(1) { }
    ~U_Ptr() { delete p; }

    int count;
    T *p; //实际数据存放
};
```



实现自己的引用计数

```
template <typename T>
class SmartPtr { //智能指针
    U_Ptr<T> *rp;
public:
    SmartPtr(T *ptr) :rp(new U_Ptr<T>(ptr)) { }
    SmartPtr(const SmartPtr<T> &sp) :rp(sp.rp) {
        ++rp->count;
    }
    SmartPtr& operator=(const SmartPtr<T>& rhs) {
        ++rhs.rp->count;
        if (--rp->count == 0) //减少自身所指rp的引用计数 pA = pB
            delete rp; //删除所指向的辅助指针
        rp = rhs.rp;
        return *this;
    }
    ~SmartPtr() {
        if (--rp->count == 0)
            delete rp;
    }
}
```

实现自己的引用计数

```
T & operator *() { return *(rp->p); }
T* operator ->() { return rp->p; }
};

int main(int argc, char *argv[]) {
    int *pi = new int(2);
    SmartPtr<int> ptr1(pi); //构造函数
    SmartPtr<int> ptr2(ptr1); //拷贝构造
    SmartPtr<int> ptr3(new int(3)); //能否ptr3(pi)???
    ptr3 = ptr2; //注意赋值运算
    cout << *ptr1 << endl; //输出2
    *ptr1 = 20;
    cout << *ptr2 << endl; //输出20

    return 0;
}
```

shared_ptr的其他用法

■ 其他用法

- `p.get()` 获取裸指针
- `p.reset()` 清除指针并减少引用计数
- `static_pointer_cast<int>(p)`
转为int类型指针(和static_cast类似, 无类型检查)
- `dynamic_pointer_cast<Base>(p)`
转为int类型指针(和dynamic_cast类似, 动态类型检查)

■ 注意!

- 不能使用同一裸指针初始化多个智能指针

```
int* p = new int();  
shared_ptr<int> p1(p); shared_ptr<int> p2(p);  
// 会产生多个辅助指针!
```
- 不能直接使用智能指针维护对象数组
(为什么?)

智能指针不总是智能

■ Parent类和Child类

```
#include <memory>
#include <iostream>
using namespace std;
```

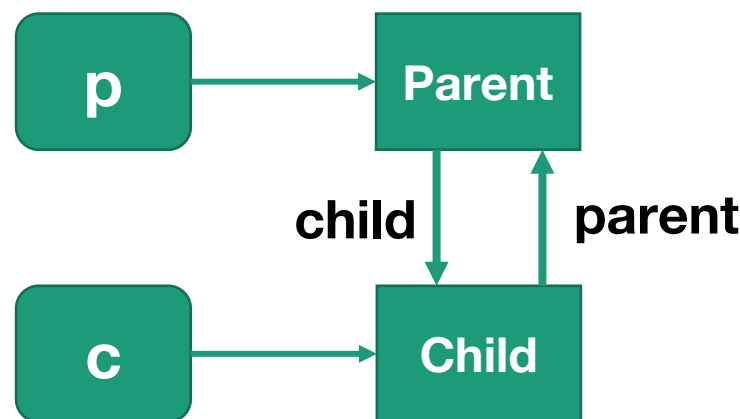
```
class Child;
class Parent {
    shared_ptr<Child> child;
public:
    Parent() {cout << "parent constructing" << endl; }
    ~Parent() {cout << "parent destructing" << endl; }
    void setChild(shared_ptr<Child> c) {
        child = c;
    }
};
class Child {
    shared_ptr<Parent> parent;
public:
    Child() {cout << "child constructing" << endl; }
    ~Child() {cout << "child destructing" << endl; }
    void setParent(shared_ptr<Parent> p) {
        parent = p;
    }
};
```

智能指针不总是智能

■ Parent类和Child类

```
void test() {  
    shared_ptr<Parent> p(new Parent());  
    shared_ptr<Child> c(new Child());  
    p->setChild(c);  
    c->setParent(p);  
    //p和c被销毁  
}
```

```
int main()  
{  
    test();  
    return 0;  
}
```



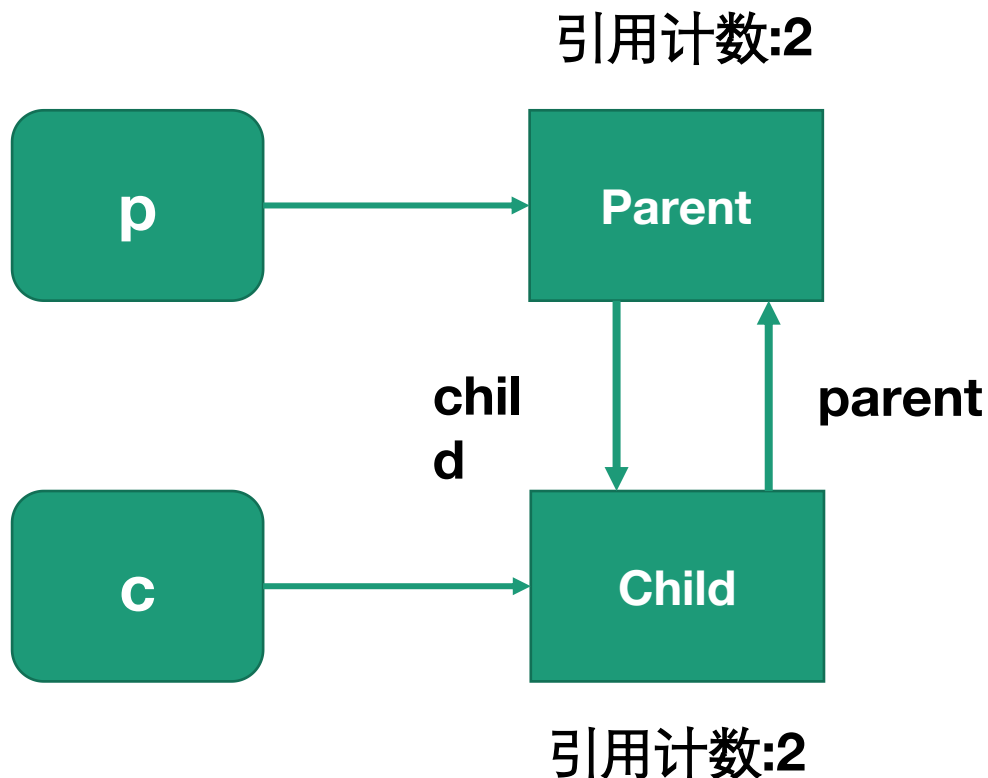
输出结果：

parent constructing
child constructing

没有析构?

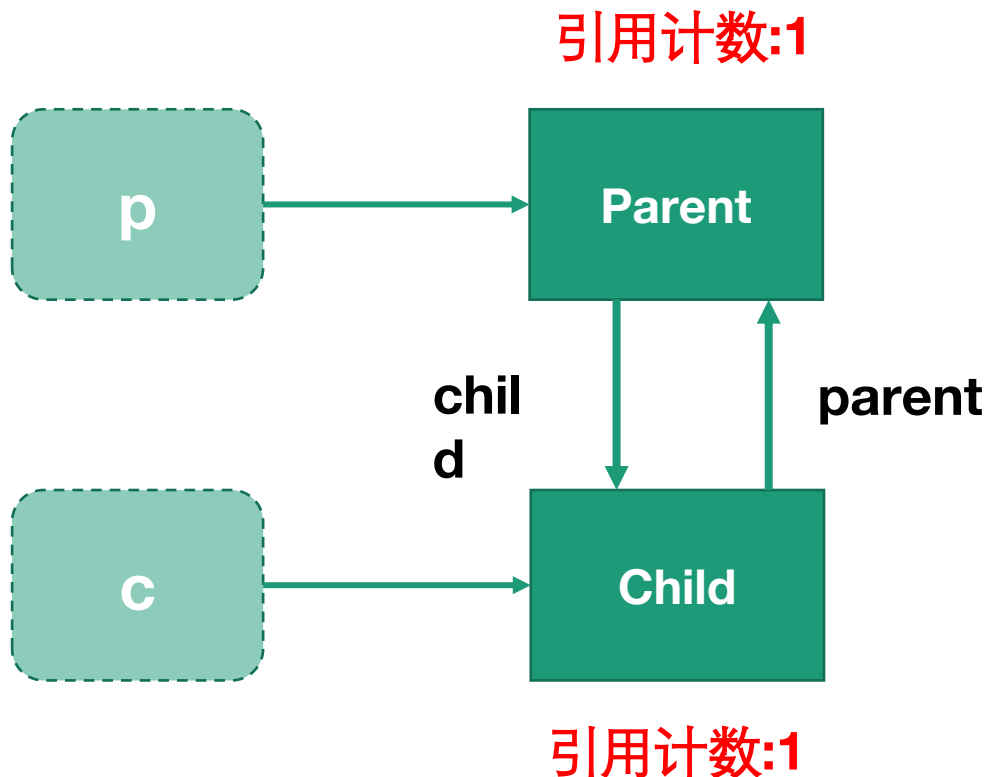
智能指针不总是智能

■ 为什么？



智能指针不总是智能

■ 为什么？



■ 两个对象的引用次数都是1，内存泄漏！

智能指针不总是智能

■ 修改Child类

```
class Child {  
    weak_ptr<Parent> parent; //修改为弱引用  
public:  
    Child() {cout << "child constructing" << endl; }  
    ~Child() {cout << "child destructing" << endl; }  
    void setParent(shared_ptr<Parent> p) {  
        parent = p;  
    }  
};
```

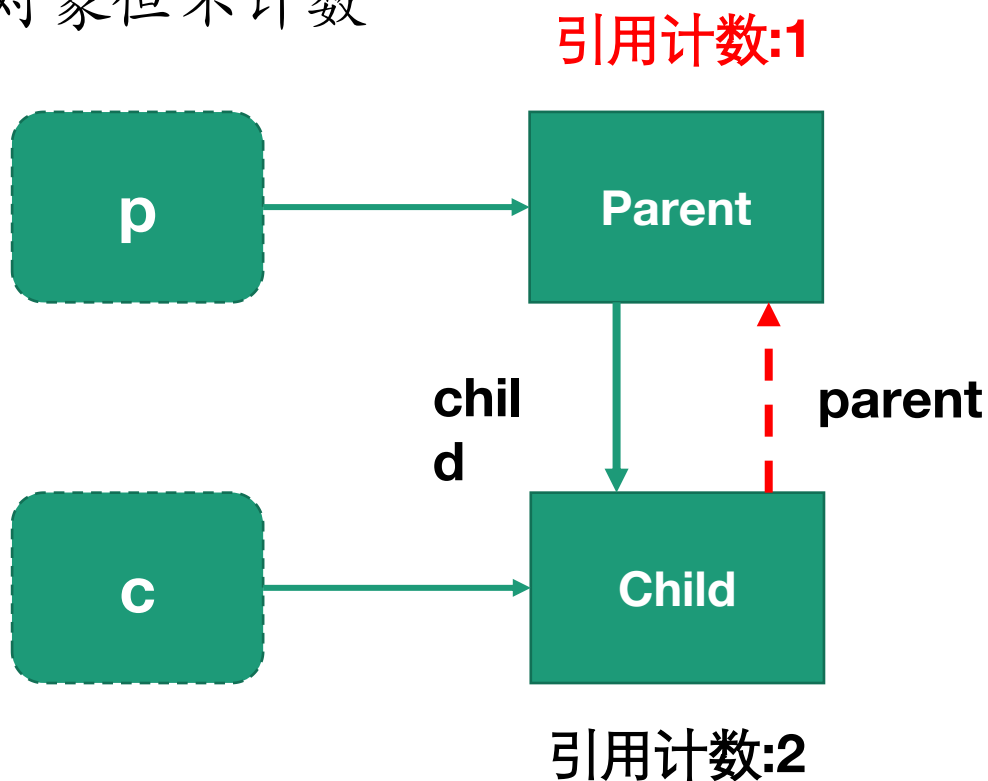
输出结果:

```
parent constructing  
child constructing  
parent destructing  
child destructing
```

智能指针不总是智能

■ 弱引用 `weak_ptr`

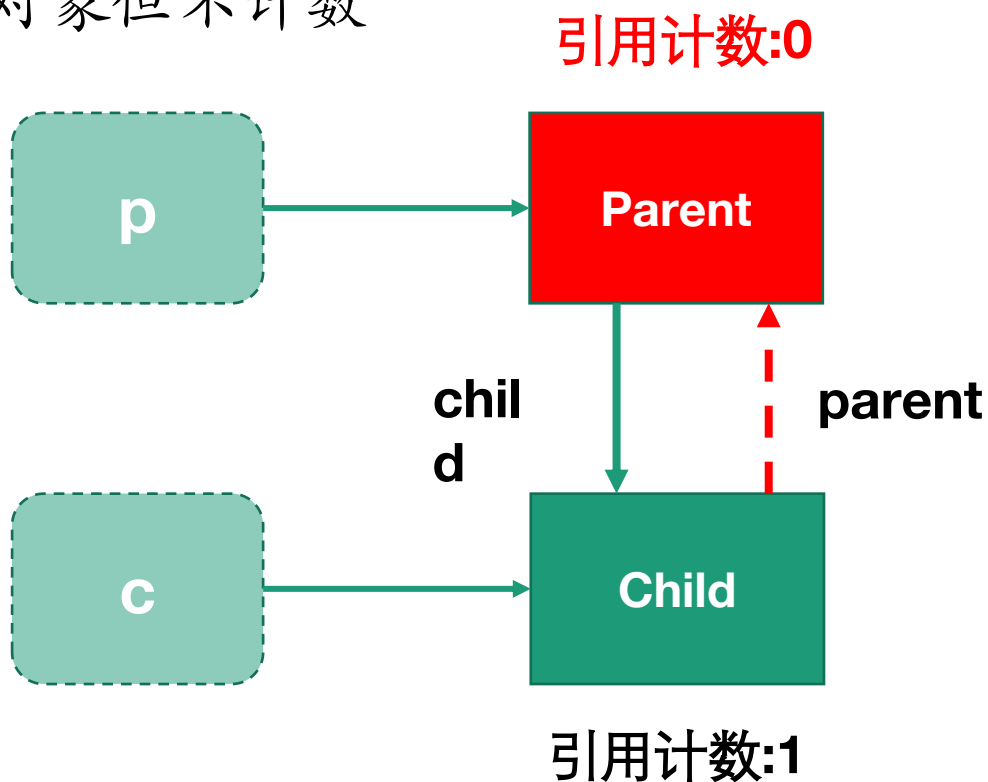
- 指向对象但不计数



智能指针不总是智能

■ 弱引用 `weak_ptr`

- 指向对象但不计数

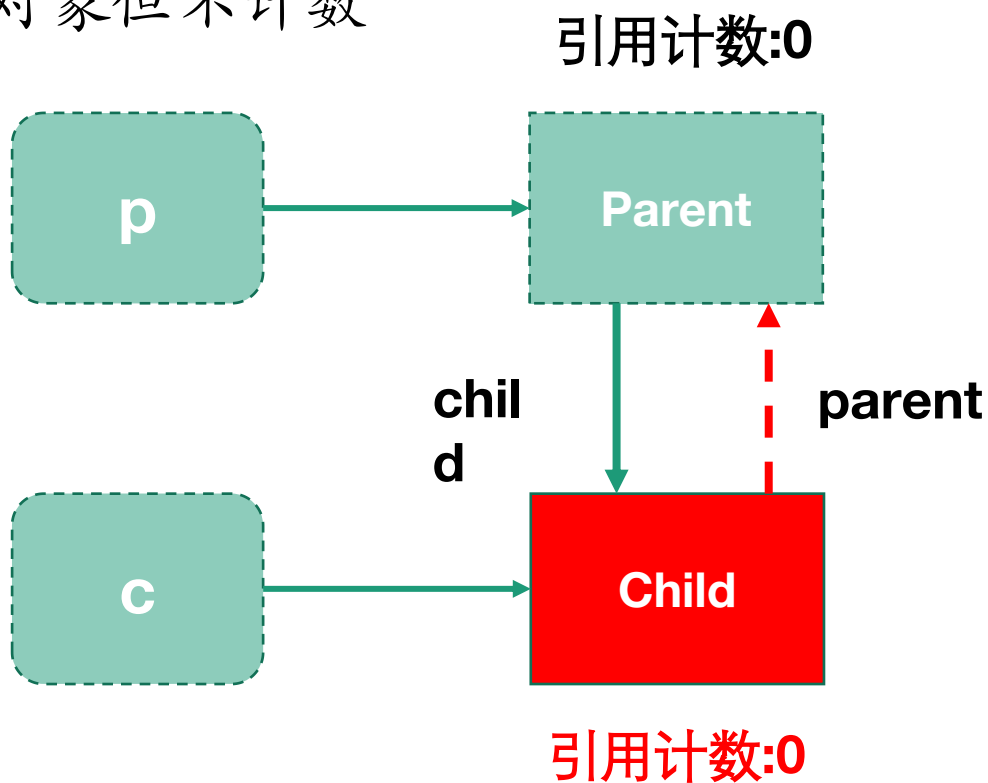


- **Parent**即将销毁

智能指针不总是智能

■ 弱引用 `weak_ptr`

- 指向对象但不计数



- Child即将销毁

弱引用

■ 弱引用指针的创建

```
shared_ptr<int> sp(new int(3));  
weak_ptr<int> wp1 = sp;
```

■ 弱引用指针的用法

```
wp.use_count() //获取引用计数  
wp.reset()      //清除指针  
wp.expired()    //检查对象是否无效  
sp = wp.lock()  //从弱引用获得一个智能指针
```

例子：弱引用

```
#include <memory>
#include <iostream>
using namespace std;
int main()
{
    std::weak_ptr<int> wp;
    {
        auto sp1 = std::make_shared<int>(20);
        wp = sp1;
        cout << wp.use_count() << endl;    //1
        auto sp2 = wp.lock();    //从弱引用中获得一个shared_ptr
        cout << wp.use_count() << endl;    //2
        sp1.reset();    //sp1释放指针
        cout << wp.use_count() << endl;    //1
    }    //sp2销毁
    cout << wp.use_count() << endl;    //0
    cout << wp.expired() << endl;    //检查弱引用是否失效: True
    return 0;
}
```

独享所有权

- `shared_ptr` 涉及引用计数，性能较差
- 如果要保证一个对象只被一个指针引用
- `unique_ptr`

```
#include <memory>
#include <utility>
using namespace std;
int main() {
    auto up1 = std::make_unique<int>(20);
    //unique_ptr<int> up2 = up1;
    //错误，不能复制unique指针
    unique_ptr<int> up2 = std::move(up1);
    //可以移动unique指针
    int* p = up2.release();
    //放弃指针控制权，返回裸指针

    delete p;
    return 0;
}
```

智能指针总结

■ 优点

- 智能指针可以帮助管理内存，避免内存泄露
- 区分unique_ptr和shared_ptr能够明确语义
- 在手动维护指针不可行，复制对象开销太大时，智能指针是唯一选择。

■ 缺点

- 引用计数会影响性能
- 智能指针不总是智能，需要了解内部原理
- 需要小心环状结构和数组指针

结束

字符串处理与 正则表达式（自学）

用户名注册

■ 场景：用户名注册

- 只能包含小写字母、数字、下划线，并且限制用户名长度在3~15个字符之间

合法例子： john_123

非法例子： John_123 / jo / @john

- 如何处理？

```
bool check(string name){  
    if (name.length() < 3 || name.length() > 15) return false;  
    for(char c: name){  
        if(!((c >= 'a' && c <= 'z') || //小写字母  
            (c >= '0' && c <= '9') || //数字  
            c == '_')) return false;  
    }  
    return true;  
}
```

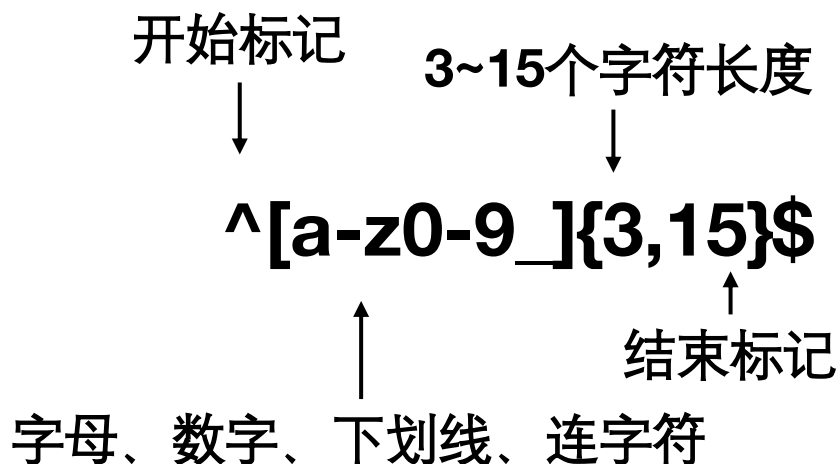
太过复杂，不易修改

正则表达式

■ 正则表达式：由字母和符号组成的特殊文本，搜索文本时定义的一种规则

■ 场景：用户名注册

- 只能包含小写字母、数字、下划线和连字符，并且限制用户名长度在3~15个字符之间
- 使用正则表达式表示规则



正则表达式

■ 正则表达式的三种模式

- 匹配：判断整个字符串是否满足条件

`^[a-z0-9_]{3,15}$`

能与john_123匹配，不能与Jo匹配

- 搜索：符合正则表达式的子串

在"q123e456w"找出所有数字串 **`[0-9]+`**

搜索结果 123, 456

- 替换：按规则替换字符串的子串

给定"q123e456w"将所有数字串替换为(number)

替换结果 q(123)e(456)w

如何编写正则表达式？

字符簇

■ 字符代表其本身

- 如：使用the进行搜索，可以找到句中所有的"the"
The car parked in **the** garage.

■ 匹配的单个字符在某个范围中

- [a-z] 匹配所有单个小写字母
- [0-9] 匹配所有单个数字

字符簇

■ 连用

- `[a-z][0-9]` 匹配所有字母+数字的组合，比如a1、b9
- `[Tt]he: The car parked in the garage.`

■ 特殊字符

- `\S` 匹配所有非空白字符
- `\d` 等价`[0-9]`，匹配所有单个数字
- `\w` 匹配字母、数字、下划线，等价`[a-zA-Z0-9_]`
- `.` 匹配除换行以外任意字符
 - `.ar: The car parked in the garage.`
- `\.` 可表示匹配句号
 - `ge\.: The car parked in the garage.`

重复模式

■ $x\{n,m\}$ 代表前面内容出现次数重复 $n\sim m$ 次

- $a\{4\}$ 匹配aaaa
- $a\{2,4\}$ 匹配aa、aaa、aaaa
- $a\{2, \}$ 匹配长度大于等于2的a

■ 特殊字符

- + 前一个字符至少连续出现1次及以上
 - $a\backslash w^+$: The car parked in a garage.
- * 前一个字符至少连续出现0次及以上
 - $a\backslash w^*$: The car parked in a garage.

正则表达式辅助工具

■ 动态匹配

- 可以反复测试
- 以染色区分匹配部分

常用正则表达式

中文字符 双字节字符 空白行 Email地址

正浮点数 腾讯QQ号 邮政编码 IP 身

`(\w+\.){2}\w+`

☐ 不区分大小写 ☐ 对^\$前后换行也支持 ☐ 符号匹配所有

`mails.tsinghua.edu.cn`

匹配到 1 条结果:

`mails.tsinghua.edu`

<http://tool.chinaz.com/regex/>

正则表达式库 <regex>

■如何在C++中使用正则表达式

- <regex>库

■创建一个正则表达式对象

- `regex re("^[1-9][0-9]{10}$")` 11位数
- 注意：C++的字符串中"`\`"也是转义字符
 - 如果需要创建正则表达式"`\d+`"，应该写成
 - `regex re("\\d+")`

原生字符串

■ 原生字符串

- 原生字符串可以取消转义，保留字面值
- 语法： **R"(str)"** 表示str的字面值
- `"\\d+" = R"(\d+)" = \d+`
- 原生字符串能换行，比如
`string str = R"(Hello
World)";`
- 结果 `str = "hello\nWorld"`

正则表达式库 <regex>

■ 匹配

- **regex_match**(s, re): 询问字符串s是否能完全匹配正则表达式re

```
#include <iostream>
#include <string>
#include <regex>
using namespace std;

int main() {
    string s("subject");
    regex e("sub.*");
    if(regex_match(s,e))
        cout << "matched" << endl;
    return 0;
}
```

输出: **matched**

捕获和分组

■ 有时我们想要获取匹配每一个部分的细节

- 例如：在 `\w*\d*` 中，我们想知道 `\w*`和`\d*`分别匹配了什么

■ 使用()进行标识，每个标识的内容被称作分组

- 正则表达式匹配后，每个分组的内容将被捕获
- 用于提取关键信息，例如`version(\d+)`即可捕获版本号

正则表达式库 <regex>

■ 捕获和分组

- `regex_match(s, result, re)`: 询问字符串s是否能完全匹配正则表达式re, 并将捕获结果储存到result中
- result需要是smatch类型的对象

```
#include <iostream>
#include <string>
#include <regex>
using namespace std;
int main () {
    string s("version10");
    regex e(R"(version(\d+))"); smatch sm;
    if(regex_match(s,sm,e)) {
        cout << sm.size() << " matches\n";
        cout << "the matches were:" << endl;
        for (unsigned i=0; i<sm.size(); ++i) {
            cout << sm[i] << endl;
        }
    }
    return 0;
}
```

输出:

2 matches
the matches were:
version10
10

捕获和分组

■ 分组会按顺序标号

- 0号永远是匹配的字符串本身
- (a)(pple): 0号为apple, 1号为a, 2号为pple
- 用(sub)(.*)匹配subject: 0号为subject, 1号为sub, 2号为ject

正则表达式库 <regex>

■ 搜索

- `regex_search(s, result, re)`: 搜索字符串s中能够匹配正则表达式re的**第一个**子串，并将结果存储在result中
- result是一个smatch对象
- 对于该子串，分组同样会被捕获

搜索的例子

```
#include <iostream>
#include <string>
#include <regex>
using namespace std;
```

```
int main() {
    string s("this subject has a submarine");
    regex e(R"((sub)([\\S]*))");
    smatch sm;
    //每次搜索时当仅保存第一个匹配到的子串
    while(regex_search(s,sm,e)){
        for (unsigned i=0; i<sm.size(); ++i)
            cout << "[" << sm[i] << "]" ";
        cout << endl;
        s = sm.suffix().str();
    }
    return 0;
}
```

输出:

```
[subject] [sub] [ject]
[submarine] [sub] [marine]
```

正则表达式库 <regex>

■ 替换

- `regex_replace(s, re, s1)`: 替换字符串s中**所有**匹配正则表达式re的子串，并替换成s1
- s1可以是一个普通文本

替换的例子

```
#include <iostream>
#include <string>
#include <regex>
using namespace std;

int main() {
    string s("this subject has a submarine");
    regex e(R"(sub[\S]*)");
    //regex_replace返回值即为替换后的字符串
    cout << regex_replace(s,e,"SUB") << "\n";
    return 0;
}
```

输出：
this SUB has a SUB

正则表达式库 <regex>

■ 替换

- `regex_replace(s, re, s1)`: 替换字符串s中**所有**匹配正则表达式re的子串，并替换成s1
- s1也可以使用一些**特殊符号**，代表捕获的分组
 - `$&` 代表re匹配的子串
 - `$1, $2` 代表re匹配的第1/2个分组

替换的例子

```
#include <iostream>
#include <string>
#include <regex>
using namespace std;
```

```
int main() {
    string s("this subject has a submarine");
    regex e(R"((sub)([\\S]*))");
    //regex_replace返回值即为替换后的字符串
    cout << regex_replace(s,e,"SUBJECT") << endl;
    //${}&表示所有匹配成功的部分, [${}&]表示将其用[]括起来
    cout << regex_replace(s,e,"[${}&]") << endl;
    //${i}输出e中第i个括号匹配到的值
    cout << regex_replace(s,e,"$1") << endl;
    cout << regex_replace(s,e,"$2") << endl;
    cout << regex_replace(s,e,"$1 and [${2}]" << endl;
    return 0;
}
```

输出:

```
this SUBJECT has a SUBJECT
this [subject] has a [submarine]
this sub has a sub
this ject has a marine
this sub and [ject] has a sub and [marine]
```

例题：学生信息整理

从多个同学的自我介绍中分别提取以下信息：名字、出生年月、电话号码、邮箱。这些自我介绍满足以下特点：

- 姓名：大家都会以“I am xxx.”或者“My name is xxx.”的语句开头。
- 出生日期：yyyy-mm-dd、yyyy.mm.dd。
- 电话：11位数字，不以0开头。
- 邮箱：由数字、字母、@、. 组成，不含其它字符。介绍中只会包含一个邮箱。

样例输入

I am **zhangshuaishuai**. I was born on **2000.10.2**. My phone number is **18866667777** and you can also reach me by my email: **zhangss@tsinghua.edu.cn**

```

void extract(string input) {
    smatch sm;
    regex get_name(R"((My name is | am )(\w+)\.))");
    regex get_date(R"((\d{4})[\.-](\d{1,2})([\.-](\d{1,2})))");
    regex get_mobile(R"([1-9]\d{10})");
    regex get_email(R"([\w]+@(\w+\.)+\w+)");

    if (std::regex_search(input, sm, get_name))
        cout << sm[2] << endl;
    int date[3] = {0};
    if (regex_search(input, sm, get_date)){
        for (int i = 1; i <= 3; i++)
            date[i - 1] = stoi(sm[i]);
        cout << date[0] << "." << date[1] << "." <<
            date[2] << endl;
    }
    if (regex_search(input, sm, get_mobile))
        cout << sm[0] << endl;
    if (regex_search(input, sm, get_email))
        cout << sm[0] << endl;
}

```

```
#include <iostream>
#include <regex>
#include <string>
using namespace std;
void extract(string input);
int main() {
    string str("I am zhangshuaishuai. \
               I was born on 2000.10.2. \
               My phone number is 18866667777 and you can also \
               reach me by my email: zhangss@tsinghua.edu.cn");
    extract(str);
    return 0;
}
```

输出

zhangshuaishuai

2000.10.2

18866667777

zhangss@tsinghua.edu.cn

字符簇（自学）

■ 转义字符

- `\n`表示换行、`\t`表示制表符

■ 范围取反

- `[^a-z]`: 匹配所有非小写字母的单个字符
- `[^c]ar`: The car **parked** in the **garage**.
- `^[^0-9][0-9]$`: 匹配长度为2的内容，且第一个不为数字，第二个为数字

■ 特殊字符

- `\D` 等价`[^0-9]`，匹配所有单个非数字
- `\s` 匹配所有空白字符，如`\t`,`\n`
- `\S` 匹配所有非空白字符
- `\W` 匹配非字母、数字、下划线，等价`[^a-zA-Z0-9_]`
- `^`代表字符串开头，`$`代表字符串结尾
 - 如: `^\t`只能匹配到以制表符开头的内容
 - 如: `^bucket$`只能匹配到只含bucket的内容

重复模式（自学）

■ $x\{n,m\}$ 代表前面内容出现次数重复 $n\sim m$ 次，可扩展到字符簇

- $[a-z]\{5,12\}$ 代表为长度为5~12的英文字母组合
- $\{5\}$ 所有长度为5的字符

■ 特殊字符

- ? 出现0次或1次
 - $[T]?he$: **The** car parked in **the** garage.
- + 至少连续出现1次及以上
 - $c.+e$: The **car parked in the garage.**
- * 至少连续出现0次及以上
 - $[a-z]^*$: **The car parked in the garage.**

或连接符（自学）

■ 匹配模式可以使用'|'进行连接

- (Chapter|Section) [1-9][0-9]?
可以匹配Chapter 1、Section 10等
- 0\d{2}-\d{8}|0\d{3}-\d{7}
可以匹配010-12345678、0376-2233445
- (c|g|p)ar: The **car** parked in the **gar**age.

■ 使用()改变优先级

- m|food 可以匹配 m 或者 food
- (m|f)ood 可以匹配 mood 或者 food
- (T|t)he|car: **The car** parked in **the** garage.

捕获和分组（自学）

■ 分组会按顺序标号

- 0号永远是匹配的字符串本身
- (a)(pple): 0号为apple, 1号为a, 2号为pple
- 用(sub)(.*)匹配subject: 0号为subject, 1号为sub, 2号为ject

■ 如果需要括号，又不想捕获该分组，可以使用(?:pattern)

- 用(?:sub)(.*)匹配subject: 0号为subject, 1号为ject

更多内容（自学）

■ 预查

- 正向预查(?=pattern) (?!pattern)
- 反向预查(?<=pattern) (?<!=pattern)

■ 后向引用

- `\b(\w+)\b\s+\1\b` 匹配重复两遍的单词
- 比如 `go go` 或 `kitty kitty`

■ 贪婪与懒惰

- 默认多次重复为贪婪匹配，即匹配次数最多
- 在重复模式后加`?` 可以变为懒惰匹配，即匹配次数最少

拓展阅读

- 智能指针来自于<memory>库，负责对动态内存管理的封装

- <http://www.cplusplus.com/reference/memory/>

- 主要包含四个部分

- Allocators 内存创建
 - Manage Pointers 智能指针，本节内容
还包含一些辅助智能指针使用的函数
 - Uninitialized Memory 对未初始化内存的操作
 - Memory Model 其他内存管理

拓展自学

■ 以下哪些能正常编译?

```
int func1(const int &x, int &b) {...};  
function<_____> pf1 = func1;
```

- A. `int(int, int)`
- B. `int(int, int&)`
- C. `int(int&, int&)`
- D. `int(const int&, int&)`
- E. `int(const int, int)`
- F. `int&(int, int&)`

拓展自学

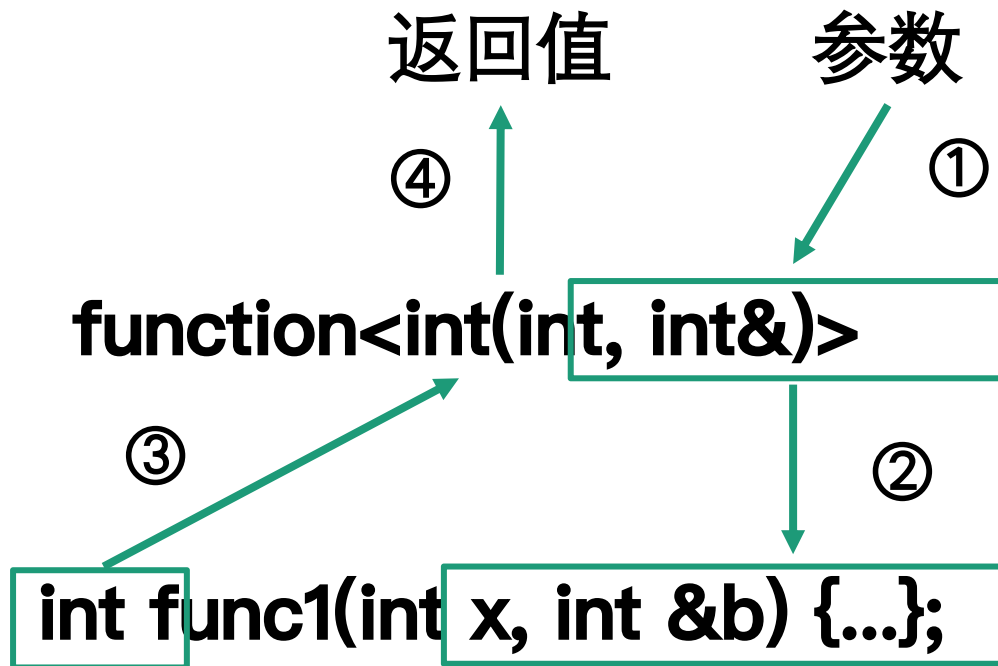
■ 以下哪些能正常编译?

```
int func1(const int &x, int &b) {...};  
function<_____> pf1 = func1;
```

- A. `int(int, int)`
- B. `int(int, int&)`
- C. `int(int&, int&)`
- D. `int(const int&, int&)`
- E. `int(const int, int)`
- F. `int&(int, int&)`

拓展自学

■ 考虑调用时的顺序



准则：
function参数 比 实际参数 更严格
function返回值 比 实际返回值 更宽松

拓展自学

绑定优先级

<https://github.com/thu-coai/THUOOP/issues/4>

■ 严格和宽松如何判定?

	左值	常量左值
	lvalue	const lvalue
int	√	√
int&	√	
const int&	√	√

若实际参数为 **int**，可绑定两种情况
function参数为**int&**，只可绑定左值
则**int&**比**int**更严格

拓展自学

■ 准则应用

```
int func1(const int &x, int &b) {...};  
function<_____> pf1 = func1;
```

A. **int(int, int)**

第一参数：int和const int&范围一致

第二参数：int比int&更宽松 因此错误

返回值：int和int范围一致

B. **int(int, int&)**

第一参数：int和const int&范围一致

第二参数：int&和int&范围一致

返回值：int和int范围一致

拓展自学

■ 准则应用

```
int func1(const int &x, int &b) {...};  
function<_____> pf1 = func1;
```

C. `int(int&, int&)`

第一参数: `int&`比`const int&`严格

第二参数: `int&`和`int&`范围一致

返回值: `int`和`int`范围一致

D. `int(const int&, int&)`

所有参数都一致

拓展自学

■ 准则应用

```
int func1(const int &x, int &b) {...};  
function<_____> pf1 = func1;
```

E. `int(const int, int)`

第一参数: `const int`比`const int&`严格

第二参数: `int`比`int&`宽松, 因此错误

返回值: `int`和`int`范围一致

F. `int&(int, int&)`

第一参数: `int`和`const int&`范围一致

第二参数: `int&`和`int&`范围一致

返回值: `int&`比`int`严格, 因此错误

拓展自学

准则：**function参数** 比 **实际参数** **更严格**
function返回值 比 **实际返回值** **更宽松**

■ 这个准则的原因是什么？

- 所有能被function接受的参数，都应被实际函数接受
- 所有可能的实际函数的返回值，都可能被function返回
- 不满足这个条件的后果？

```
int func1(const int &x, int &b) {...};  
function<int(int, int)> pf1 = func1;
```

pf1(x, b); //b可能会被func1修改?和pf1声明不符

拓展自学

■ 以下哪些能正常编译?

```
class Func2{  
public:  
    int& operator()(int &&b) const {...}  
};  
function<_____> pf2 = Func2();
```

- A. int&(int&&)
- B. int(int&&)
- C. int&(int&)
- D. int(int&)

拓展自学

■ 以下哪些能正常编译?

```
class Func2{  
public:  
    int& operator()(int &&b) const {...}  
};  
function<_____> pf2 = Func2();
```

A. `int&(int&&)`

B. `int(int&&)`

C. `int&(int&)`

D. `int(int&)`

■ 关于右值的判定

	左值	常量左值	右值	常量右值
	lvalue	const lvalue	rvalue	const rvalue
int	√	√	√	√
int&	√			
const int&	√	√	√	√
int&&			√	
const int&&			√	√

一般不使用 `const int&&`，无实际意义

大家可以参照此表，用准则验证上一页答案

拓展自学

■ 例外

```
class Func2{  
public:  
    int& operator()(int &&b) const {...}  
};  
function<_____> pf2 = Func2();
```

请自己尝试： `int&(int)` 是否可以？

拓展自学

■ 例外

```
class Func2{  
public:  
    int& operator()(int &&b) const {...}  
};  
function<int&(int)> pf2 = Func2();
```

例外：当实际参数为T&&时，function参数可以为T

pf2的在内部拷贝了一份参数，然后将该参数的右值传给Func2()